Performance Testing with Grafana K6

This lab is individual homework. Work through it at home, then submit the link to your GitHub repository on MyCourses. Check the deadline there.

You will write K6 scripts, run them against QuickPizza, a public demo site that Grafana runs for K6 practice, and read the results the way a performance tester does. Part 0 teaches the basics step by step. Part A builds a realistic user journey and runs it as a smoke, average-load, stress, spike and soak test. Part B asks you to explain what you saw.

1. Learning outcomes

This lab trains six skills.

1. Write a K6 script that sends HTTP requests and checks each response.
2. Control the load with virtual users, duration, iterations and stages.
3. Read the K6 summary, in particular the request rate, the p(95) of `http_req_duration` and the failure rate.
4. Turn a performance requirement into thresholds that make a test pass or fail.
5. Write smoke, average-load, stress, spike and soak tests, and compare their results.
6. Explain the results in terms of the operational envelope of a system.

2. Before you start

Tools	K6, Git, a GitHub account, and Visual Studio Code or another editor with a terminal
Test site	QuickPizza at https://quickpizza.grafana.com, a public demo site for K6 practice

2.1 Install K6

Install K6 with the command for your operating system.

On macOS, with Homebrew

```
brew install k6
```

On Windows, with winget

```
winget install k6 --source winget
```

On Windows, with Chocolatey if you use it instead

```
choco install k6
```

On Windows without a package manager, download and run the installer from https://dl.k6.io/msi/k6-latest-amd64.msi.

On Ubuntu or Debian

```
K=/usr/share/keyrings/k6-archive-keyring.gpg
curl -fsSL https://dl.k6.io/key.gpg | sudo gpg --dearmor -o $K
echo "deb [signed-by=$K] https://dl.k6.io/deb stable main" \
  | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt-get update
sudo apt-get install k6
```

Then check the installation. The command prints the version of K6. This sheet was tested with k6 v2.2.0.

```
k6 version
```

2.2 Create your repository

Keep all your work for this lab in one public GitHub repository named `k6-lab`, so that the link you submit opens for the instructor.

1. Sign in at github.com and open github.com/new. Name the repository `k6-lab`, choose Public, tick Add a README file, and create it.
2. In Visual Studio Code, open the Command Palette, run Git: Clone, paste the HTTPS address of the repository, and choose a folder outside OneDrive, iCloud Drive and Google Drive.
3. Open the cloned folder and open a terminal in it with Terminal, New Terminal. Run every command in this sheet from that folder.
4. Create a folder named `reports` in it.
5. Commit and push after each step with the three commands below, changing the message each time.

```
git add .
git commit -m "Step 0.1"
git push
```

2.3 Rules for QuickPizza

QuickPizza is shared by everyone in the world who learns K6, and Grafana offers it for small tests. Follow these rules.

- Never run more than 5 virtual users against QuickPizza.
- Keep each run to 3 minutes or less. The soak test in Step A5 is the one exception, and it runs only 2 virtual users.
- Never run a breakpoint test, or any test with hundreds of users, against a site that you do not own or have no written permission to test.

Part 0: K6 basics

Step 0.1: Your first test GUIDED

Create `01-first-test.js` with the code below.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export default function () {
  const res = http.get('https://quickpizza.grafana.com/');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

The default function is what one virtual user does again and again. Here it opens the home page, checks that the status code is 200, and sleeps for one second, which stands for the thinking time of a real user. Run it.

```
k6 run 01-first-test.js
```

With no options, K6 runs one virtual user once. The summary at the end shows what happened. Find these lines in it.

- `checks_succeeded` shows 100.00% 1 out of 1, and a tick stands next to `status is 200`.
- `http_req_duration` gives the response time as avg, min, med, max, p(90) and p(95). With one request, all of them are the same number.
- `http_reqs` is 1, the number of HTTP requests, followed by the rate per second.
- `iterations` is 1, the number of times the default function ran.

Write the `http_req_duration` of this run in RESULTS.md. Section 4 gives a template for that file.

Step 0.2: More users for longer GUIDED

Run the same script with 5 virtual users for 20 seconds. The options on the command line change the load without changing the script.

```
k6 run --vus 5 --duration 20s 01-first-test.js
```

When we ran it on 21 October 2026, it made 78 requests at about 3.7 requests per second, with a p(95) of about 290 ms. Your numbers will differ a little.

Write your three numbers in RESULTS.md, and explain why the rate stays below 5 requests per second although 5 users run for the whole 20 seconds. Look at what one virtual user does in one iteration, and how long that takes.

Step 0.3: A fixed number of iterations GUIDED

Now ask for exactly 20 iterations, shared by the 5 virtual users.

```
k6 run --vus 5 --iterations 20 01-first-test.js
```

`iterations` is now exactly 20, and the run ends as soon as the 20th iteration finishes. A duration gives load over a period of time, while a number of iterations gives an exact amount of work.

Step 0.4: Testing an API GUIDED

QuickPizza recommends a pizza through its API. A POST request to `/api/pizza` carries the customer's restrictions as JSON and needs a token in the Authorization header. The token `abcdef0123456789` is the public one that QuickPizza gives for practice. Create `04-pizza-api.js`.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://quickpizza.grafana.com';
const PARAMS = {
  headers: {
    'Content-Type': 'application/json',
    Authorization: 'token abcdef0123456789',
  },
};

export default function () {
  const body = JSON.stringify({ mustBeVegetarian: true });
  const res = http.post(`${BASE_URL}/api/pizza`, body, PARAMS);
  check(res, {
    'status is 200': (r) => r.status === 200,
    'pizza has a name': (r) => r.json('pizza.name') !== undefined,
    'every ingredient is vegetarian': (r) =>
      r.json('pizza.ingredients').every((i) => i.vegetarian === true),
  });
  sleep(1);
}
```

Run it.

```
k6 run 04-pizza-api.js
```

All three checks should pass. The third check looks inside the answer. `r.json('pizza.ingredients')` reads a field of the JSON body, and the check confirms that the API honoured the vegetarian restriction. A load test that checks only status codes can miss an API that answers quickly but wrongly.

Step 0.5: A request that must be refused GUIDED

A good API refuses a request that has no valid token. Create `05-no-token.js`, which sends the same request without a token and expects status 401.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://quickpizza.grafana.com';

export default function () {
  const body = JSON.stringify({ mustBeVegetarian: true });
  const res = http.post(`${BASE_URL}/api/pizza`, body, {
    headers: { 'Content-Type': 'application/json' },
    responseCallback: http.expectedStatuses(401),
  });
  check(res, {
    'request without a token is refused': (r) => r.status === 401,
  });
  sleep(1);
}
```

Run it.

```
k6 run 05-no-token.js
```

K6 normally counts every response with status 400 or above as a failed request in `http_req_failed`. The line with `http.expectedStatuses(401)` tells K6 that 401 is the expected answer here, so `http_req_failed` stays at 0.00% while the check passes. This is a small security test inside a performance tool.

Step 0.6: Thresholds GUIDED

Checks count passes and failures, but they never make the whole test fail. Thresholds do. A threshold is a pass or fail rule on a metric, and it turns a performance requirement into a test result.

Create `06-thresholds.js` with the code below.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://quickpizza.grafana.com';
const PARAMS = {
  headers: {
    'Content-Type': 'application/json',
    Authorization: 'token abcdef0123456789',
  },
};
const P95 = __ENV.P95 || 1500;

export const options = {
  vus: 3,
  duration: '30s',
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: [`p(95)<${P95}`],
    checks: ['rate>0.99'],
  },
};

export default function () {
  const body = JSON.stringify({ mustBeVegetarian: true });
  const res = http.post(`${BASE_URL}/api/pizza`, body, PARAMS);
  check(res, {
    'status is 200': (r) => r.status === 200,
    'pizza has a name': (r) => r.json('pizza.name') !== undefined,
  });
  sleep(1);
}
```

The three thresholds say that fewer than 1% of requests may fail, that 95% of requests must finish within the number in `P95`, which is 1,500 ms unless you set it, and that more than 99% of checks must pass. Run it.

```
k6 run 06-thresholds.js
```

The THRESHOLDS block at the top of the summary shows a tick for each rule that passed. Now make the response time rule stricter without editing the script.

```
k6 run -e P95=300 06-thresholds.js
```

The p(95) rule now shows a cross, K6 prints that the thresholds on `http_req_duration` have been crossed, and the exit code becomes 99. Check the exit code right after the run. On macOS and Linux, type the first line below. In Windows PowerShell, type the second.

```
echo $?
$LASTEXITCODE
```

A pipeline such as GitHub Actions reads this exit code and fails the build, so a change that slows the system down is caught the same way as a failing unit test. Write both exit codes in RESULTS.md.

Step 0.7: From a requirement to thresholds YOUR TURN

A product owner gives you this requirement.

> *With 3 users sending requests at the same time for 30 seconds, the pizza recommendation API answers 95% of requests within 1 second and 99% within 1.5 seconds, and fewer than 1% of requests fail.*

1. Create `07-requirement.js`. Start from `06-thresholds.js`, and set `vus`, `duration` and the thresholds so that the test checks exactly this requirement. One metric can carry more than one rule, for example `['p(95)<1000', 'p(99)<1500']`.
2. Run it, and write in RESULTS.md which rules passed, with the measured p(95) and p(99).
3. Change one number in the requirement so that the test fails, run it again, and write down the exit code.

Step 0.8: Bug hunt checkpoint YOUR TURN

A customer wants a pizza without pepperoni that can be cut without a knife. The script below asks QuickPizza for exactly that, and two checks look at the answer. Create `08-bug-hunt.js`.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL = 'https://quickpizza.grafana.com';
const PARAMS = {
  headers: {
    'Content-Type': 'application/json',
    Authorization: 'token abcdef0123456789',
  },
};

// The customer wants no pepperoni, and no knife to cut the pizza.
export default function () {
  const body = JSON.stringify({
    excludedIngredients: ['pepperoni'],
    excludedTools: ['knife'],
  });
  const res = http.post(`${BASE_URL}/api/pizza`, body, PARAMS);
  check(res, {
    'status is 200': (r) => r.status === 200,
    'no pepperoni': (r) =>
      !r.json('pizza.ingredients').some((i) => i.name === 'Pepperoni'),
    'no knife needed': (r) => r.json('pizza.tool') !== 'Knife',
  });
  sleep(1);
}
```

1. Run it with 5 virtual users for 20 seconds.

```
k6 run --vus 5 --duration 20s 08-bug-hunt.js
```

2. Some checks fail, and the summary shows how often. Every failed check is a pizza that the customer did not ask for.
3. Find out why. Explore the API with single requests. Create `explore.js` below and run it with `k6 run explore.js`. It prints the toppings and the tools that QuickPizza knows.

```
import http from 'k6/http';

const BASE_URL = 'https://quickpizza.grafana.com';
const PARAMS = { headers: { Authorization: 'token abcdef0123456789' } };

export default function () {
  const toppings = http.get(`${BASE_URL}/api/ingredients/topping`, PARAMS);
  console.log(toppings.body);
  const tools = http.get(`${BASE_URL}/api/tools`, PARAMS);
  console.log(tools.body);
}
```

4. Fix the request in the script, run it again, and confirm that every check passes.
5. Explain in RESULTS.md what was wrong, and why checks under load found it when one manual request might not have.

3. The ideas you are about to practice

3.1 Load test types as stages

In Part A, one script describes a user journey, and each test type runs the same journey with different stages. The table shows the shapes in this lab, scaled down for a public site, next to the examples in the Grafana K6 documentation.

Test type	What it asks	Stages in this lab	Example in the K6 documentation
Smoke	Does the script work at all?	1 user for 30 seconds	A few users for a short time
Average load	How does the system behave under normal traffic?	Up to 3 users in 30 s, 2 min at 3, down in 30 s	Up to 100 users in 5 min, 30 min at 100, down in 5 min
Stress	How does it behave above the normal load?	Up to 5 users in 30 s, 2 min at 5, down in 30 s	Up to 200 users in 10 min, 30 min at 200, down in 5 min
Spike	Does it survive a sudden jump, and how fast does it recover?	1 user, a jump to 5 in 5 s, 30 s at 5, back to 1 for 1 min	Up to 2,000 users in 2 min, down to 0 in 1 min
Soak	Does it get worse over a long time?	2 users for 10 min	100 users for 8 hours
Breakpoint	Where does it break?	Not run in this lab	More and more users until the system fails

3.2 Think time

A real user reads a page before the next click. The `sleep` calls in the journey model that pause. Without them, every virtual user would send requests back to back, and 5 virtual users would produce far more traffic than 5 real people.

3.3 Percentiles

An average hides the slowest users. The p(95) of a run is the time that 95% of its requests beat, so 1 request in 20 takes longer. Requirements and thresholds therefore use percentiles rather than averages.

3.4 The operational envelope

A system promises to work within an operational envelope, for example fast answers up to a certain load. Tests near the edge of the envelope check that promise, and tests well beyond it check that the system slows down gracefully rather than failing (Pezzè and Young, Software Testing and Analysis, Section 22.2). On a shared public site you can test only far inside the envelope, and Part B asks you to think about what that means for your results.

Part A: A user journey and the load test types

Step A1: The user journey and a smoke test GUIDED

Create `journey.js`. It describes one visit to QuickPizza in three groups, and every test in Part A imports it.

```
import http from 'k6/http';
import { check, group, sleep } from 'k6';

const BASE_URL = 'https://quickpizza.grafana.com';
const PARAMS = {
  headers: {
    'Content-Type': 'application/json',
    Authorization: 'token abcdef0123456789',
  },
};

// One visit is the home page, the quotes, then a vegetarian pizza.
export function journey() {
  group('home', () => {
    const res = http.get(`${BASE_URL}/`);
    check(res, { 'home page is 200': (r) => r.status === 200 });
  });
  sleep(1 + Math.random() * 2);

  group('quotes', () => {
    const res = http.get(`${BASE_URL}/api/quotes`);
    check(res, {
      'quotes are listed': (r) =>
        r.status === 200 && r.json('quotes').length > 0,
    });
  });
  sleep(1);

  group('recommendation', () => {
    const body = JSON.stringify({ mustBeVegetarian: true });
    const res = http.post(`${BASE_URL}/api/pizza`, body, PARAMS);
    check(res, {
      'recommendation is 200': (r) => r.status === 200,
      'pizza has a name': (r) => r.json('pizza.name') !== undefined,
    });
  });
  sleep(1 + Math.random() * 2);
}
```

Groups name the parts of the journey, so that the results of each part can be told apart. `Math.random()` varies the think time between 1 and 3 seconds, so that the users do not move in lockstep. Now create `a1-smoke.js`, which runs the journey with 1 user for 30 seconds.

```
import { journey } from './journey.js';

export const options = {
  vus: 1,
  duration: '30s',
  thresholds: {
    checks: ['rate>0.99'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  journey();
}
```

```
k6 run a1-smoke.js
```

Every check should pass. When a smoke test fails, the script is usually wrong, so fix it before you run any bigger test.

Step A2: The average-load test and an HTML report GUIDED

Create `a2-average.js`. It ramps up to 3 users, holds them for 2 minutes and ramps down.

```
import { journey } from './journey.js';

export const options = {
  stages: [
    { duration: '30s', target: 3 },
    { duration: '2m', target: 3 },
    { duration: '30s', target: 0 },
  ],
  thresholds: {
    checks: ['rate>0.99'],
    http_req_failed: ['rate<0.01'],
    'http_req_duration{group:::home}': ['p(95)<1000'],
    'http_req_duration{group:::recommendation}': ['p(95)<1500'],
  },
};

export default function () {
  journey();
}
```

The last two thresholds use the group tag, so the home page and the recommendation each get their own response time rule. Run the test, and save an HTML report of the run at the same time.

```
k6 run --out web-dashboard=export=reports/a2-average.html a2-average.js
```

At the start of the run, K6 prints the address of a live dashboard, http://127.0.0.1:5665, which you can open in a browser while the test runs. When the test ends, open `reports/a2-average.html`. Its Overview section plots the request rate, the p(95) of the response time, the number of virtual users and the request duration over the whole run. Fill in the row for Step A2 in the results table in RESULTS.md.

Step A3: A stress test YOUR TURN

Copy `a2-average.js` to `a3-stress.js` and change the stages. Ramp up to 5 users, the most allowed on QuickPizza, over 30 seconds, hold them for 2 minutes, then ramp down over 30 seconds. Run it with a report as in Step A2, with `a3-stress` in the file name of the report. Compare the p(95) of `http_req_duration` with Step A2 in the results table.

Step A4: A spike test YOUR TURN

Write `a4-spike.js`. Start with 1 user for 30 seconds, jump to 5 users within 5 seconds, hold them for 30 seconds, drop back to 1 user within 5 seconds, stay at 1 user for 1 minute, and end with 10 seconds of ramp down. Run it with a report, as in Step A2. In the report, find the moment of the spike. Did the response time rise during the spike, and how long did it take to return to its earlier level?

Step A5: A soak test YOUR TURN

Write `a5-soak.js`. Ramp up to 2 users in 30 seconds, hold them for 10 minutes, and ramp down in 30 seconds. Run it with a report, as in Step A2. A real soak test runs for hours, which you must not do against a public site. In the report, compare the response time in the first minutes with the last minutes. Did the response time or the error rate get worse as the test went on?

Part B: Reflection

Answer these questions in RESULTS.md, with the numbers from your runs.

1. Fill in the results table for Steps A1 to A5.
2. In Step 0.2, why was the request rate below 5 requests per second with 5 virtual users?
3. Did the response time grow from the average-load test to the stress test? Explain your answer with the operational envelope.
4. In the spike test, how long did the response time take to return to normal after the spike?
5. In the soak test, did the response time or the error rate get worse over time? Name one kind of defect that a soak test of several hours could reveal.
6. Why do the thresholds in this lab use p(95) rather than the average?
7. Your results come from a shared public site reached over the internet. Give two reasons why the same tests against a system in a lab could give different numbers.
8. Why must you never run a stress, spike or breakpoint test with hundreds of users against a site that you do not own?
9. Write one vague performance requirement for an app you use every day. Rewrite it as a testable requirement, and show the K6 thresholds that check it.

4. Deliverables

Push everything to your `k6-lab` repository, and submit its link on MyCourses before the deadline shown there. The repository should hold every script from Part 0 and Part A, including your fixed bug

hunt script, the HTML reports of Steps A2 to A5, and `RESULTS.md` with your notes from Part 0, the results table and your answers to Part B. It should look like this.

```
k6-lab/
  README.md
  RESULTS.md
  01-first-test.js
  04-pizza-api.js
  05-no-token.js
  06-thresholds.js
  07-requirement.js
  08-bug-hunt.js
  explore.js
  journey.js
  a1-smoke.js
  a2-average.js
  a3-stress.js
  a4-spike.js
  a5-soak.js
  reports/
    a2-average.html
    a3-stress.html
    a4-spike.html
    a5-soak.html
```

Start RESULTS.md from this template. In the table, Users is the highest number of virtual users, Req/s is the rate printed next to `http_reqs`, p(95) and Max come from `http_req_duration`, Failed is `http_req_failed`, and Checks is `checks_succeeded`.

```
# K6 lab results

## Part 0
- Step 0.1, http_req_duration:
- Step 0.2, requests, rate and p(95), and why the rate stays below 5:
- Step 0.6, exit code of the normal run and of the run with P95=300:
- Step 0.7, rules that passed, p(95), p(99), exit code after your change:
- Step 0.8, what was wrong, and how the checks showed it:

## Part A results
| Step | Test type    | Users | Req/s | p(95) | Max | Failed | Checks |
| ---- | ------------ | ----- | ----- | ----- | --- | ------ | ------ |
| A1   | Smoke        | 1     |       |       |     |        |        |
| A2   | Average load | 3     |       |       |     |        |        |
| A3   | Stress       | 5     |       |       |     |        |        |
| A4   | Spike        | 5     |       |       |     |        |        |
| A5   | Soak         | 2     |       |       |     |        |        |

## Part B
1.
2.
3.
4.
5.
6.
7.
8.
9.
```